ISS 2103 • EXAM REVISION PACK 07

# Transport & Application Layers

Course outline weeks 8–11: transport design issues, session & presentation layers, connection management, RPC, compression, FTP, e-mail, virtual terminals + lab commands

Priority: **MEDIUM–HIGH**

**TCP vs UDP** has been asked in 4 of the 5 papers (it is the only part of this topic on the ISA2024 paper). The 2018 paper put a whole 25-mark question on DNS, troubleshooting and TCP. Outline topics such as RPC, compression, FTP and e-mail have *not* been asked yet, so they are covered here and in the practice questions.

## How often this topic is examined

| Question asked | Paper(s) | Marks |
|---|---|---|
| TCP vs UDP (differences at layer 4) | 2018 Q2a, SE2024 Q5e | 4–5 |
| How data is sent with UDP (connectionless) | 2023 Q5c, ISA2024 Q5a | 4 |
| Why TCP is called reliable (with example) | 2018 Q5a | 6 |
| TCP 3-way handshake with diagrams | 2018 Q3c | 6 |
| How DNS works, recursive lookups | 2018 Q2b | 10 |
| Troubleshooting a PC that cannot reach the Internet | 2018 Q2c | 10 |
| Services the Internet offers (list four) | SE2024 Q5c | 4 |
| RPC, compression, FTP, e-mail, Telnet/SSH, cables | not yet asked (course outline) | likely |

**EXAM TIP**

Every TCP question gets more marks with a **time-line diagram** (two vertical lines, slanted arrows). Practise drawing the handshake and the lost-segment diagram in under 2 minutes each.

## 1 Transport layer services, ports and sockets

The transport layer gives **process-to-process** (end-to-end) delivery of the whole message. The network layer (IP) only gets a packet to the right *computer*; the transport layer gets it to the right *program* on that computer.

- **Segmentation and reassembly**: the message is cut into **segments** carrying sequence numbers, so the receiver can put them back in order.
- **Port address** (service-point address): a 16-bit number (0–65 535) that labels a process. Well-known ports 0–1023 (HTTP 80, Telnet 23), registered 1024–49 151, dynamic/ephemeral 49 152–65 535 (chosen by the client).
- **Socket** = IP address + port, e.g. 196.4.70.10:80. A TCP connection is identified by the pair of sockets (client socket, server socket).
- **Multiplexing** / **demultiplexing**: many applications share one IP address; the port numbers sort the incoming segments to the right process.
- **Connection control** (set-up and release), **flow control** and **error control**: done *end to end*, not link by link as at the data link layer.
- Transport protocols in TCP/IP: **TCP** (reliable, connection-oriented), **UDP** (simple, connectionless), **SCTP** (combines both; used for VoIP signalling).

**Transport layer design issues (outline weeks 8–9)**

Addressing (ports, sockets) • connection establishment • connection release • flow control and buffering • multiplexing many conversations over one network connection • error control • crash recovery.

| Service | Port | Service | Port |
|---|---|---|---|
| FTP data / control | TCP 20 / 21 | HTTP / HTTPS | TCP 80 / 443 |
| SSH | TCP 22 | POP3 / IMAP | TCP 110 / 143 |
| Telnet | TCP 23 | DNS | UDP 53 (TCP 53 zone transfer) |
| SMTP | TCP 25 (587) | DHCP server / client | UDP 67 / 68 |
| TFTP | UDP 69 | SNMP | UDP 161 / 162 |

**Port** identifies the program, **IP** the host, **MAC** the network card. Three addresses, three layers.

## 2 TCP vs UDP

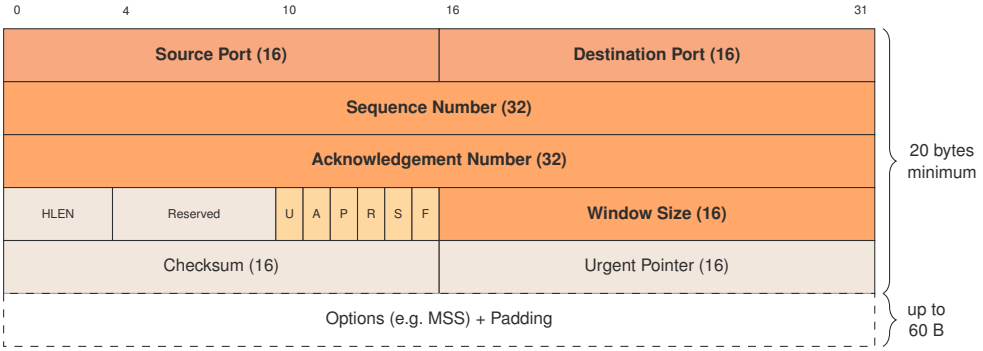



Figure: TCP header. Flags: URG, ACK, PSH, RST, SYN, FIN. Orange = fields that make TCP reliable.

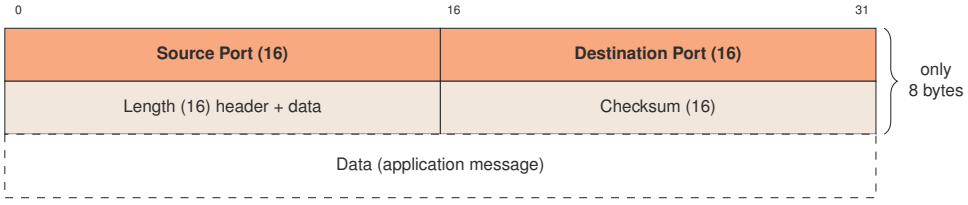



Figure: UDP header. No sequence number, no ACK, no window: that is why it is so small.

**PAST PAPER** **2018 Q2a / SE2024 Q5e** **[4–5 marks]**

Differentiate between TCP and UDP as they apply to layer four (4) of the OSI model. / Discuss the differences between TCP and UDP.

**Model answer**

| Feature | TCP (Transmission Control Protocol) | UDP (User Datagram Protocol) |
|---|---|---|
| **Connection** | Connection-oriented: 3-way handshake before data, 4-way close | Connectionless: no set-up, just sends datagrams |
| **Reliability** | Reliable: sequence numbers, ACKs, retransmission of lost segments | Unreliable (best effort): no ACKs, no retransmission |
| **Ordering** | Delivers bytes in order (reorders by sequence number) | No ordering; datagrams may arrive out of order |
| **Flow / congestion control** | Yes: sliding window, slow start | None |
| **Header** | 20–60 bytes | 8 bytes |
| **Speed** | Slower, more overhead | Faster, low overhead, low delay |
| **Data unit** | Segment (byte stream) | Datagram (separate messages) |
| **Casting** | Unicast only | Unicast, broadcast and multicast |
| **Used by** | HTTP, FTP, SMTP, SSH, Telnet | DNS, DHCP, SNMP, TFTP, VoIP, streaming, games |

**REMEMBER IT**

**TCP = a phone call**: dial, the other side answers, you talk in order, you say goodbye. **UDP = posting a letter**: drop it in the box and hope; no reply that it arrived.

## 3 How UDP sends data

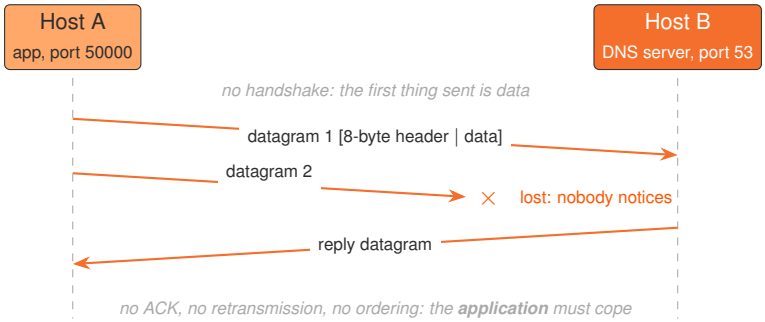



Figure: UDP connectionless transmission.

**PAST PAPER** **2023 Q5c / ISA2024 Q5a** **[4 marks]**

In contrast to TCP, UDP is described as a connectionless protocol. Briefly explain how data is transmitted between two computers using the UDP protocol.

**Model answer**

1. The application passes a message to UDP, which adds an **8-byte header** (source port, destination port, length, checksum) to form a **user datagram**.
2. There is **no connection set-up**: the datagram is handed straight to IP, which routes it on its own to the destination IP address.
3. Each datagram is **independent**: datagrams may take different paths and arrive out of order, duplicated or not at all.
4. The receiving UDP checks the checksum; a damaged datagram is silently discarded, otherwise the data goes to the process named by the **destination port**.

No ACKs, retransmission, flow or congestion control, so it is fast with low overhead: ideal for DNS queries, DHCP, VoIP and live video.

## 4 Why TCP is reliable

"Connectionless" does not mean "no addresses". Every UDP datagram still carries both port numbers and travels inside an IP packet with both IP addresses.

TCP gives a **reliable byte stream** over an unreliable IP network using these mechanisms:

- **Sequence numbers**: every byte is numbered, so missing, duplicate or out-of-order data is found.
- **Cumulative acknowledgements**: ACK = the number of the *next* byte expected.
- **Retransmission**: a segment not ACKed before the **retransmission timer** expires (or after 3 duplicate ACKs, "fast retransmit") is sent again.
- **Checksum**: corrupted segments are dropped and later resent.
- **Flow control (sliding window)**: the receiver advertises a window = how many bytes it can still buffer; the sender never has more than that unacknowledged.
- **Congestion control**: slow start and congestion avoidance stop the sender flooding the network.

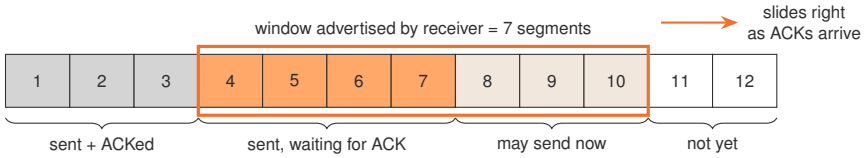



Figure: TCP sliding window (flow control).

**PAST PAPER** **2018 Q5a** **[6 marks]**

With the aid of an example explain why TCP is called a reliable protocol.

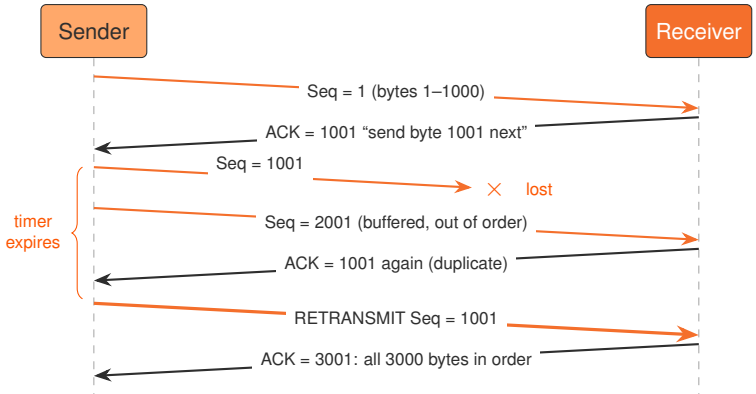



Figure: how TCP recovers a lost segment.

### Model answer

TCP is reliable because it guarantees that data arrives **complete, error-free and in order**, even though IP below it is best effort.

1. **Connection set-up**: the 3-way handshake makes sure both sides are ready and agree starting sequence numbers.
2. **Sequence numbers** on every byte let the receiver reorder segments and spot gaps or duplicates.
3. **Acknowledgements**: the receiver confirms what it got (ACK = next byte expected).
4. **Retransmission**: if no ACK comes before the timer expires, the sender resends.
5. **Checksum**: a corrupted segment is discarded and so gets resent.
6. **Flow and congestion control**: the window stops buffers overflowing, so data is not lost.

**Example** (figure): segment 1001 is lost; the receiver keeps sending ACK 1001; the sender's timer expires and it resends 1001; the receiver then ACKs 3001, so all data arrives in order.

## 5 Connection management: 3-way handshake and 4-way close

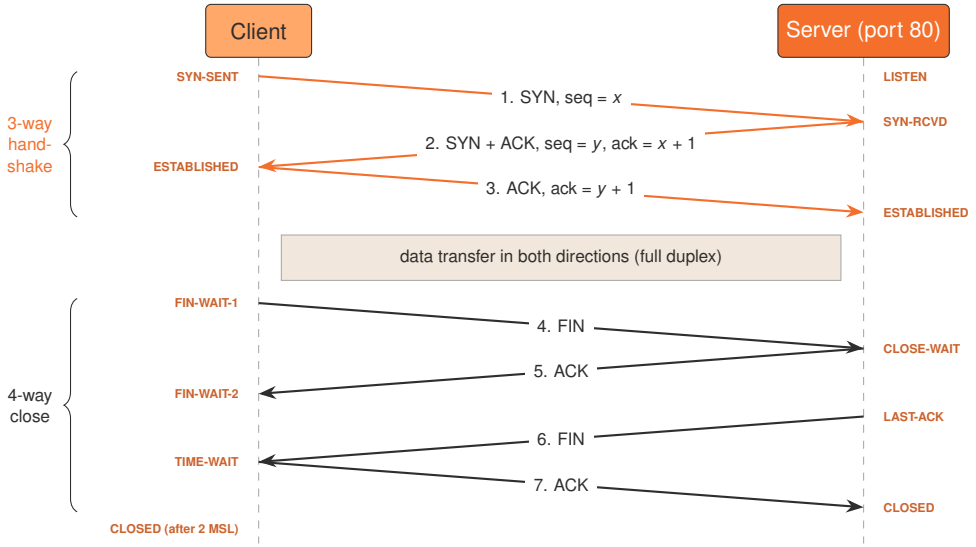



Figure: TCP connection set-up and release, with the connection state of each side.

**PAST PAPER** **2018 Q3c** **[6 marks]**

Describe the TCP 3-way handshake using diagrams.

**Model answer**

(Draw steps 1–3 of the figure above.) The handshake opens a TCP connection and synchronises sequence numbers in both directions.

1. **SYN**: the client sends a segment with the SYN flag set and its initial sequence number (ISN = $x$), asking to connect. Client state: SYN-SENT.
2. **SYN-ACK**: the server, which was LISTENing, replies with SYN and ACK set: its own ISN ($y$) and ack = $x + 1$, meaning "I got your SYN and I am ready".
3. **ACK**: the client sends ACK with ack = $y + 1$. Both sides are now ESTABLISHED and full-duplex data transfer can start.

**Why 3 steps**: each side must send its own ISN *and* have it acknowledged; it also confirms both are reachable and agrees MSS and window size. Abuse: a **SYN flood** attack sends many SYNs and never the final ACK, filling the server with half-open connections.

**Connection release (4-way close)**

TCP is full duplex, so each direction is closed **separately**: FIN → ACK one way, FIN → ACK the other way. Between steps 5 and 6 the server may still send data (half-close). The side that closes first waits in **TIME-WAIT** (2 × maximum segment lifetime) so a lost final ACK can be resent and old segments die out. A connection can also be cut at once with an **RST** (reset) segment.

The SYN and FIN flags each use up one sequence number. That is why the reply is $x + 1$, not $x$.

## 6 DNS: the Domain Name System

DNS is a **distributed, hierarchical database** that translates host names (www.hit.ac.zw) into IP addresses, and back (reverse lookup). It runs over **UDP port 53** (TCP 53 for zone transfers and large replies). No single server holds all names: each **zone** is run by its own **authoritative name server**.

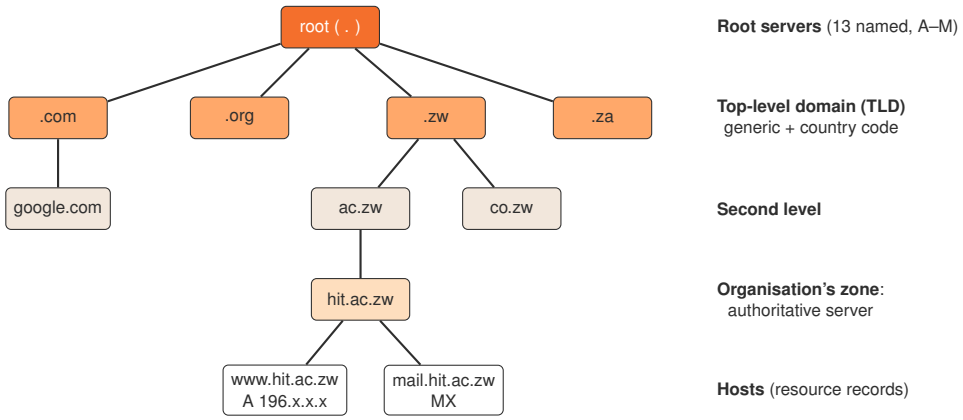



Figure: the DNS name space is an inverted tree. A full name is read from the leaf up to the root.

| Record | Holds | Record | Holds |
|---|---|---|---|
| A | IPv4 address | MX | mail server of the domain |
| AAAA | IPv6 address | NS | name server of the zone |
| CNAME | alias (another name) | PTR | IP → name (reverse) |

**Recursive vs iterative queries**

**Recursive**: the server asked must return the *final answer* (or an error); it does all the work itself. Used from the client to its local resolver.
**Iterative**: the server asked returns the best it knows, usually a *referral* ("ask this server next"), and the asker does the next query. Used from the resolver to root, TLD and authoritative servers.

**PAST PAPER** **2018 Q2b** **[10 marks]**

Describe in detail how DNS works with particular focus to recursive lookups.

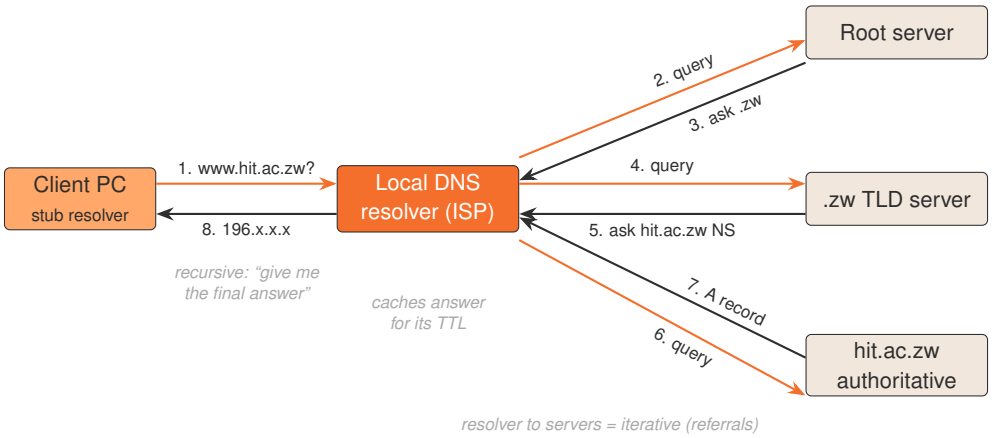



Figure: resolving www.hit.ac.zw. Orange arrows = queries, dark arrows = replies.

**Model answer**

DNS maps names to IP addresses using a hierarchy of servers: root → TLD → authoritative.

1. The user types www.hit.ac.zw. The PC first checks its own **DNS cache** and **hosts file**.
2. If not found, the **stub resolver** sends a **recursive query** (UDP 53) to its configured DNS server (ISP or company resolver). Recursive means "you do all the work and return the final answer or an error".
3. The resolver checks its **cache**; if the name is cached and its TTL has not expired, it answers at once.
4. Otherwise it asks a **root server**, which does not know the host but refers it to the **.zw TLD** servers.
5. The resolver asks the **.zw TLD server**, which refers it to the **name server for hit.ac.zw**.
6. The resolver asks the **authoritative server** for hit.ac.zw, which returns the **A record** (or AAAA for IPv6).
7. The resolver **caches** the answer for its TTL, so the next user gets it immediately.
8. It returns the IP address to the client, which can now open a TCP connection to the web server.
9. Steps 4–6 are **iterative** queries (each server gives a referral). Only the client-to-resolver query is recursive. In a *fully recursive* scheme each server would itself ask the next one and pass the answer back down the chain, which loads the root and TLD servers, so they normally refuse recursion.
10. DNS is **distributed** (no single point of failure, many root replicas) and secured against spoofing/cache poisoning by **DNSSEC**.

Always name the **three server levels**: root, TLD, authoritative. Then say which query is recursive and which are iterative.

# 7 Application layer services

## 7.1 The web (HTTP)

- **HTTP** (TCP 80) / **HTTPS** (TCP 443, HTTP over TLS): a **request–response** protocol between a browser (client) and a web server.

- Requests use methods **GET** (fetch a page), **POST** (send form data), PUT, DELETE. Replies carry a status code: 200 OK, 301 moved, 404 not found, 500 server error.
- HTTP is **stateless**: each request stands alone, so cookies/sessions are used to remember a user. Pages are named by a **URL** (protocol://host/path) and written in HTML.

## 7.2 File transfer (FTP)

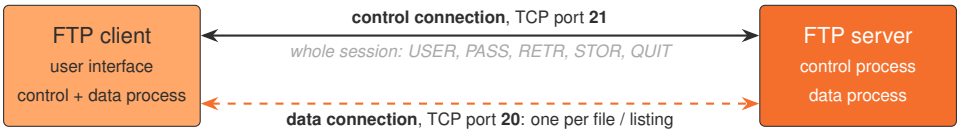



Figure: FTP uses two TCP connections ("out-of-band" control).

- **Active mode**: the server opens the data connection from its port 20 to the client. **Passive mode**: the client opens it to a port the server names (works through firewalls/NAT).
- Transfer modes: ASCII (text) and binary (image). FTP sends the password in **plain text**; secure versions are **SFTP** (over SSH) and **FTPS** (over TLS).
- **TFTP** (UDP 69): trivial FTP, no login, no listing; used to load router images and configs. **FTAM** is the OSI File Transfer, Access and Management service.

## 7.3 Electronic mail (SMTP, POP3, IMAP)

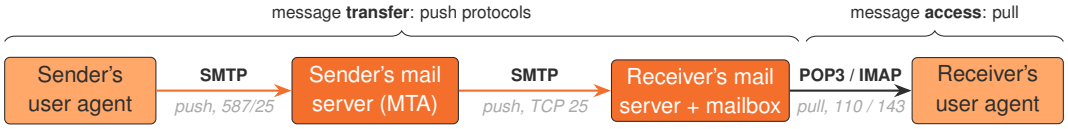



Figure: how an e-mail travels. SMTP pushes it to the mailbox; the receiver pulls it with POP3 or IMAP.

- **User agent (UA)**: the mail program (Outlook, Gmail app) used to write and read mail. **Message transfer agent (MTA)**: the mail server software that relays mail. **Mailbox**: where mail waits on the server.
- **SMTP** (TCP 25, 587 for client submission): **sends/pushes** mail from client to server and server to server. Commands HELO, MAIL FROM, RCPT TO, DATA, QUIT. 7-bit ASCII only, so **MIME** encodes attachments, images and non-English text.
- **POP3** (TCP 110, 995 secure): **downloads** mail to one device and usually **deletes** it from the server. Simple; mail can be read offline.
- **IMAP** (TCP 143, 993 secure): mail **stays on the server**; folders, read/unread flags and deletions are **synchronised** on every device; can fetch headers only.

## 7.4 Virtual terminals and remote login (Telnet, SSH)

Terminals differ (keyboard codes, screen size, line endings). A **network virtual terminal (NVT)** is a standard imaginary terminal: each side translates its local terminal to and from the NVT, so any terminal can log in to any host.

| | Telnet | SSH |
|---|---|---|
| Port | TCP 23 | TCP 22 |
| Security | Plain text: password can be sniffed | Encrypted session, server authentication, keys |
| Extra features | Remote command line only | Also file transfer (SCP/SFTP), port forwarding |
| Use today | Testing a port, old devices, labs | Remote admin of servers, routers, switches |

**PAST PAPER** **SE2024 Q5c** **[4 marks]**

The internet offers a variety of services to users. List and explain any four.

**Model answer**

1. **World Wide Web** (HTTP/HTTPS): browsing linked web pages and online services, e.g. e-learning portals.
2. **Electronic mail** (SMTP, POP3, IMAP): sending and receiving messages and attachments to any mailbox in the world.
3. **File transfer** (FTP/SFTP): uploading and downloading files between computers, e.g. updating a website.
4. **Remote login** (Telnet/SSH): using a distant computer's command line as if sitting at it.

Others: DNS (name look-up), VoIP and video calls, instant messaging, streaming, cloud storage, e-commerce/online banking.

# 8 Session and presentation layers: RPC and compression

**Session layer** (the "network dialog controller"): sets up, manages and ends sessions; **dialog control** (who talks when, half or full duplex); **synchronisation** by inserting checkpoints, so a 100 MB transfer that fails at 90 MB restarts from the last checkpoint, not from zero. **Presentation layer**: **translation** (ASCII/EBCDIC/Unicode, byte order), **encryption** and **compression**.

## 8.1 Remote Procedure Call (RPC)

RPC lets a program call a procedure on a remote machine **as if it were a local call**. The network details are hidden by two **stubs**.

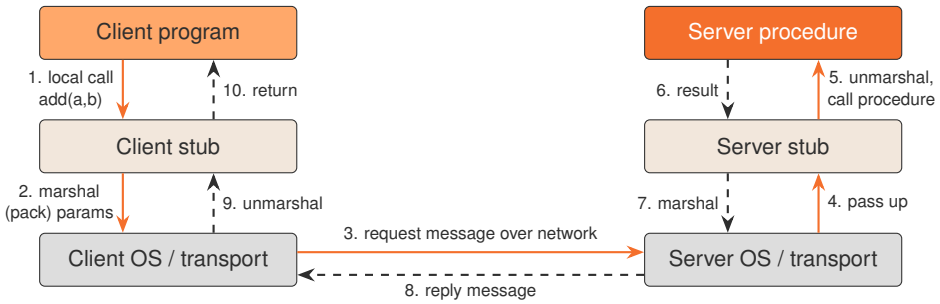



Figure: an RPC call (solid) and its return (dashed). The client is blocked until step 10.

- **Marshalling**: packing the procedure name and parameters into a message in a standard format (e.g. XDR), so machines with different data formats understand it.
- The call is normally **synchronous**: the client waits. Problems: no shared memory (no pointers), and the network or server may fail, so calls need timeouts and retries.
- Examples: Sun RPC (used by NFS), Java RMI, gRPC, Microsoft RPC/DCOM.

## 8.2 Data compression

Compression reduces the number of bits, so transfers are faster and need less bandwidth and storage. **Lossless**: the original is recovered exactly (text, programs): RLE, Huffman, LZW (ZIP, GIF, PNG). **Lossy**: some detail the eye or ear will not miss is thrown away, for much smaller files: JPEG (images), MPEG/MP4 (video), MP3 (audio).

**Run-length encoding (RLE)**: a run of repeated symbols is replaced by a count and the symbol.

**RLE example**

AAAABBBCCDAA → 4A 3B 2C 1D 2A ← *12 symbols become 10*

ABCDEF → 1A1B1C1D1E1F ← *no runs: output doubles!*

RLE only helps when there are long runs (fax pages, simple graphics, bitmaps with large one-colour areas).

**Huffman coding**: frequent symbols get **short** codes, rare symbols **long** codes. No code is the start of another (**prefix code**), so the bit stream decodes without separators.

**Huffman worked example: 39 symbols, A 15, B 7, C 6, D 6, E 5**

Sort: E5, C6, D6, B7, A15 ← *always join the two smallest*
E5 + D6 = 11 ← *tie C6/D6: either choice is valid*
C6 + B7 = 13
11 + 13 = 24
A15 + 24 = 39 (root)
Codes: A = 0, D = 100, E = 101, C = 110, B = 111 ← *left 0, right 1*
Bits = 15(1) + 6(3) + 5(3) + 6(3) + 7(3) = 87 ← *frequency × length*
Fixed 3-bit code = 39 × 3 = 117; ASCII = 39 × 8 = 312
Average = 87/39 = 2.23 bits/symbol
Saving vs 3-bit code = 30/117 ≈ 26%

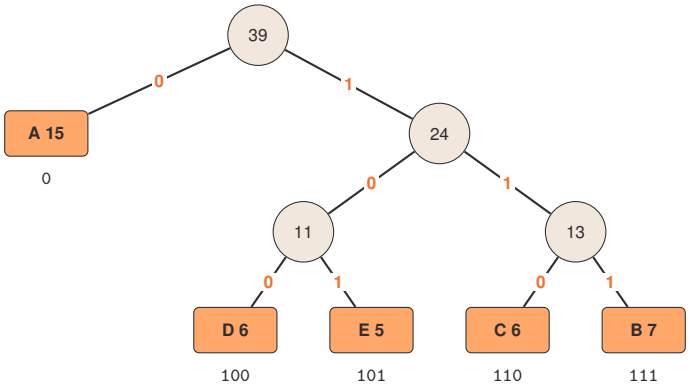



Figure: Huffman tree for the example. The most frequent symbol (A) is nearest the root.

**LZW (Lempel–Ziv–Welch)**: a **dictionary** method. It starts with single characters, adds every new string it meets to the dictionary, and sends the dictionary *index* instead of the string. The receiver rebuilds the same dictionary, so the dictionary is never sent. Used in GIF, TIFF and Unix `compress`.

Huffman needs the **frequency table** (or the tree) sent with the data, or the receiver cannot decode.

**LZW example: encode ABABABA, start dictionary A = 1, B = 2**

$w$ = A, next B : AB new ⇒ output 1, add AB = 3 ← *output the longest known string*
$w$ = B, next A : BA new ⇒ output 2, add BA = 4
$w$ = AB, next A : ABA new ⇒ output 3, add ABA = 5 ← *AB was already known*
$w$ = ABA, end ⇒ output 5
Result: 1 2 3 5 (7 symbols → 4 codes)

## 9 Troubleshooting, lab commands and cabling

| Command | What it does |
|---|---|
| `ipconfig /all` | IP address, mask, default gateway, DNS servers, MAC, DHCP lease. `/release`, `/renew`, `/flushdns` |
| `hostname` | Shows the computer's name |
| `ping <ip\|name>` | Sends ICMP Echo Requests; shows reachability, round-trip time and packet loss |
| `tracert <ip>` | Lists every router (hop) to the destination using increasing TTL; shows where packets stop |
| `netstat -an` | Active TCP/UDP connections and listening ports (spot unknown connections) |
| `nslookup <name>` | Queries DNS directly: is the name resolved, by which server? |
| `arp -a` | ARP cache: IP-to-MAC mappings on the LAN |

**PAST PAPER** **2018 Q2c** **[10 marks]**

A co-worker calls the help desk complaining about her PC not going to the internet. Outline the procedure you would take in troubleshooting this problem.

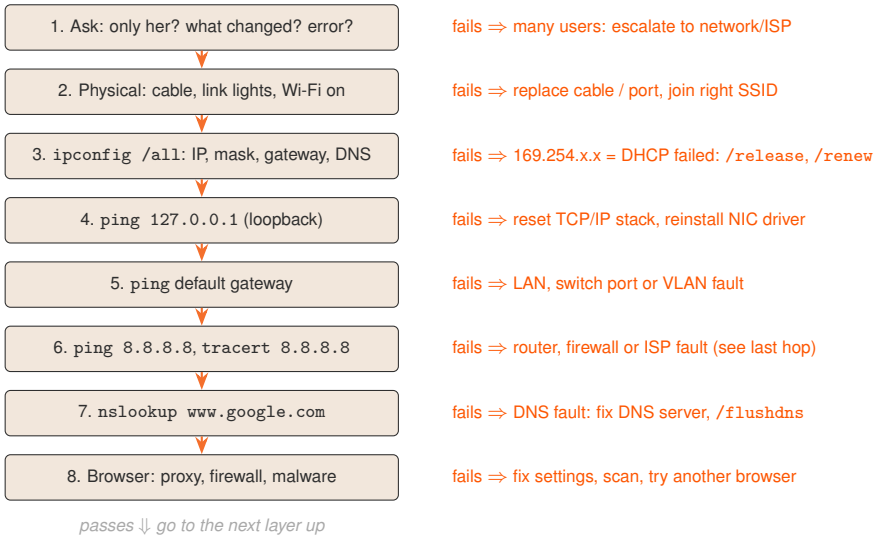



Figure: bottom-up (OSI layer 1 → 7) troubleshooting flow.

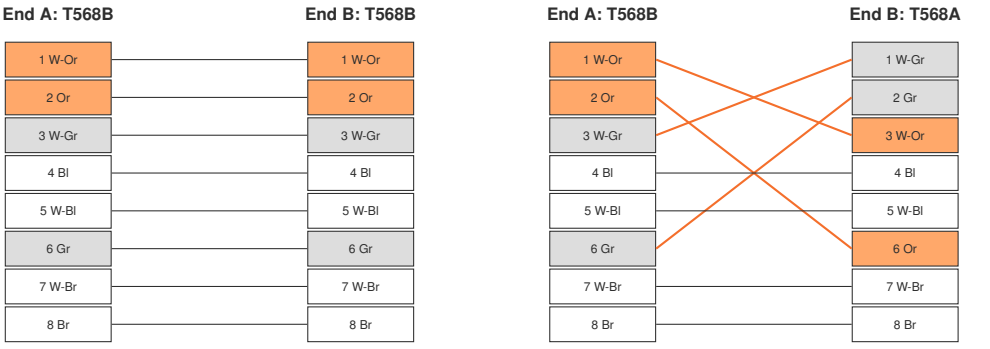
**Model answer**

Use a structured **bottom-up OSI** approach and test one thing at a time.

1. **Gather information**: what exactly fails, any error message, what changed, is it only her PC or everyone? Many users = a network/ISP problem, escalate.
2. **Physical layer**: check the cable is plugged in and NIC link lights are on, or that Wi-Fi is on and connected to the right SSID. Try another cable/port.
3. **IP configuration**: run `ipconfig /all`. Is there a valid IP, mask, gateway and DNS? A **169.254.x.x** (APIPA) address means DHCP failed: `ipconfig /release` then `/renew`, check the DHCP server.
4. **TCP/IP stack**: `ping 127.0.0.1`. If it fails, reset the stack (`netsh int ip reset`) or reinstall the NIC driver.
5. **Local network**: `ping` the default gateway. If it fails, suspect the switch port, VLAN or wrong subnet/gateway setting.
6. **Beyond the gateway**: `ping 8.8.8.8`. If it fails, run `tracert 8.8.8.8` to see which hop stops: router, firewall or ISP.
7. **DNS**: if IP addresses work but names do not, use `nslookup`; correct the DNS server and run `ipconfig /flushdns`.
8. **Application**: check browser proxy settings, firewall/antivirus rules, malware; try another browser.
9. **Compare** with a working colleague's PC to isolate the fault.
10. **Fix, verify and document**: confirm with the user that the Internet works, and record the problem and solution.

Each `ping` test tells you which *half* of the path is broken: loopback (the PC), gateway (the LAN), 8.8.8.8 (the WAN), a name (DNS).

## 9.1 UTP cables: straight-through vs crossover




**Straight-through**: pin 1 → 1 … 8 → 8
unlike devices: PC–switch, switch–router

**Crossover**: 1 ↔ 3, 2 ↔ 6 (TX to RX)
like devices: PC–PC, switch–switch, router–router, PC–router

Figure: RJ-45 pinouts. Orange pair (peach) and green pair (grey) carry data in 10/100 Mb/s Ethernet; the colour of each wire stays the same end to end.

- **Rule**: *unlike* devices (different layers) → straight-through; *like* devices → crossover. A PC and a router both transmit on pins 1–2, so they also need a crossover.
- Modern switches have **Auto-MDI/MDIX** and fix the wrong cable themselves. A **rollover (console)** cable (pins 1–8 reversed) connects a PC to a router's console port.
- **Cisco Packet Tracer**: a network simulator used in the labs to build topologies, pick cables (the lightning icon picks automatically), configure IP addresses and routers, and use **Simulation mode** to watch PDUs (ARP, ICMP, DNS, TCP) move hop by hop.

# 10 Practice questions

**PRACTICE Q1 EASY** **[3 marks]**

Give the port number and transport protocol used by: HTTP, HTTPS, SSH, SMTP, DNS, and FTP (both ports).

**Answer**
HTTP TCP 80; HTTPS TCP 443; SSH TCP 22; SMTP TCP 25; DNS UDP 53; FTP TCP 21 (control) and 20 (data).

**PRACTICE Q2 EASY** **[3 marks]**

Define a *socket* and explain why port numbers are needed at the transport layer.

**Answer**
A socket is the combination of an **IP address and a port number** (e.g. 10.0.0.5:80); a pair of sockets identifies one connection. The IP address only finds the host; many processes run on that host, so the 16-bit port number identifies the **process** that should receive the data (process-to-process delivery, multiplexing).

**PRACTICE Q3 EASY** **[4 marks]**

Distinguish between POP3 and IMAP.

**Answer**
POP3 (TCP 110) **downloads** mail to one device and usually deletes it from the server; simple, good for offline reading on one PC. IMAP (TCP 143) keeps mail **on the server** and synchronises folders, flags and deletions across all devices; can download headers only. Both are *pull* (access) protocols; SMTP is used to send.

**PRACTICE Q4 EASY** [4 marks]

State the cable type (straight-through or crossover) for: (a) PC to switch (b) switch to switch (c) PC to PC (d) router to switch.

**Answer**

(a) straight-through (b) crossover (c) crossover (d) straight-through. Rule: unlike devices straight, like devices crossover.

**PRACTICE Q5 MEDIUM** [5 marks]

Compress `WWWWWWBBBWWWWBWW` using run-length encoding. Give the compression ratio and one weakness of RLE.

**Answer**

**Working**

$$\text{WWWWWW BBB WWWW B WW} \rightarrow \text{6W 3B 4W 1B 2W} \quad \leftarrow \textit{count each run}$$

$$16 \text{ symbols} \rightarrow 10 \text{ symbols}$$

$$\text{ratio} = 16 : 10 = 1.6 : 1 \quad \leftarrow \textit{37.5\% smaller}$$

Weakness: data with few runs (e.g. normal text “ABCD”) gets **longer** (1A1B1C1D).

**PRACTICE Q6 MEDIUM** [6 marks]

Explain how a Remote Procedure Call (RPC) works. Name the layer it belongs to.

**Answer**

RPC belongs to the **session layer**. (1) The client program makes an ordinary local call to the **client stub**. (2) The stub **marshals** the procedure name and parameters into a message. (3) The client OS sends it across the network. (4) The **server stub** receives it and unmarshals the parameters. (5) It calls the real procedure on the server. (6) The result returns the same way and the client stub returns it to the program, which was blocked waiting. The programmer never sees the network. Examples: Sun RPC/NFS, Java RMI, gRPC.

**PRACTICE Q7 MEDIUM** [6 marks]

Why does FTP use two TCP connections? Distinguish active and passive FTP.

**Answer**

FTP separates **control** and **data** (out-of-band control): the control connection (port 21) stays open for the whole session carrying commands (USER, PASS, RETR, STOR) and replies, while a new **data connection** (port 20) is opened for each file or directory listing and closed afterwards. Commands can be sent (e.g. abort) while a file is transferring. **Active**: the server opens the data connection from port 20 to a port the client gives (PORT); often blocked by client firewalls/NAT. **Passive**: the client asks with PASV and opens the data connection to a port the server names; works through firewalls.

**PRACTICE Q8 MEDIUM** [6 marks]

With a diagram, explain how a TCP connection is released. Why is the close four steps, not three?

**Answer**

Draw steps 4–7 of the handshake figure. (1) The client sends **FIN** (FIN-WAIT-1). (2) The server replies **ACK** (CLOSE-WAIT); the client→server direction is now closed but the server can still send data (**half-close**). (3) When finished, the server sends its own **FIN** (LAST-ACK). (4) The

client sends the final **ACK** and waits in **TIME-WAIT** for 2 MSL so a lost ACK can be resent. Four steps because TCP is full duplex: each direction is closed separately and the server's ACK and FIN are usually sent at different times.

**PRACTICE Q9 HARD** **[12 marks]**

A message of 20 symbols has frequencies A 10, B 6, C 2, D 1, E 1. (a) Build the Huffman tree and give each code. (b) Find the total number of bits and compare with a fixed-length code and with 8-bit ASCII. (c) Decode `0101100`.

**Answer**

**Working**

D1 + E1 = 2 ← *join the two smallest*
2 + C2 = 4
4 + B6 = 10
10 + A10 = 20 (root)
Codes : A = 0, B = 10, C = 110, D = 1110, E = 1111
Bits = 10(1) + 6(2) + 2(3) + 1(4) + 1(4) = 36 ← *frequency × code length*
Fixed = 20 × 3 = 60 bits ← *5 symbols need 3 bits*
ASCII = 20 × 8 = 160 bits
Saving = $(60 - 36)/60 = 40\%$ vs fixed
Saving = $(160 - 36)/160 = 77.5\%$ vs ASCII
Average = $36/20 = 1.8$ bits/symbol

(c) `0` | `10` | `110` | `0` ⇒ **A B C A**. It decodes uniquely because no code is a prefix of another.

**PRACTICE Q10 HARD** **[10 marks]**

With the aid of a diagram, describe how an e-mail sent by a Gmail user reaches a HIT student who reads it on both a phone and a laptop. Name every protocol and component.

**Answer**

Draw the e-mail flow figure. (1) The sender writes the mail in a **user agent** (Gmail app/browser). (2) The UA **submits** it with **SMTP** (port 587, TLS) to Gmail's mail server (**MTA**). (3) The MTA does a **DNS MX lookup** for hit.ac.zw to find the receiving mail server. (4) It **relays** the message with **SMTP** (TCP 25) to HIT's mail server. (5) The HIT server stores it in the student's **mailbox**. (6) Attachments travel as **MIME**-encoded text because SMTP is 7-bit ASCII. (7) The student's phone and laptop **pull** the mail with **IMAP** (143/993). (8) IMAP is the right choice because mail stays on the server and read/deleted flags **sync on both devices**; POP3 would download it to one device and delete it. (9) SMTP is a **push** protocol, POP3/IMAP are **pull**/**access** protocols. (10) All run over TCP for reliable delivery.

**PRACTICE Q11 HARD** **[10 marks]**

(a) Encode `TOBETOBE` with LZW, starting with the dictionary B = 1, E = 2, O = 3, T = 4. (b) Compare lossless and lossy compression, giving two examples of each and a suitable use.

**Answer**

**Working**

T, next O ⇒ out 4, TO = 5 ← *TO not yet known*
O, next B ⇒ out 3, OB = 6
B, next E ⇒ out 1, BE = 7
E, next T ⇒ out 2, ET = 8
TO, next B ⇒ out 5, TOB = 9 ← *TO is in the dictionary now*
BE, end ⇒ out 7
Output: 4 3 1 2 5 7 (8 symbols → 6 codes)

(b) **Lossless**: the original is rebuilt bit for bit; lower ratio (about 2:1); needed for text, programs, bank data, medical images. Examples: Huffman, LZW (ZIP, GIF, PNG), RLE. **Lossy**: removes detail people do not notice (psycho-visual/acoustic models, DCT); high ratios (10:1 to 100:1) but the original cannot be recovered; used for photos, video and music streaming. Examples: JPEG, MPEG/MP4, MP3. Choose lossless when every bit matters and lossy when bandwidth matters more than perfect quality.

## ONE-PAGE CHEAT SHEET

**Transport layer**

- Process-to-process delivery; segments with sequence numbers
- Port = 16 bits; socket = IP + port
- Design issues: addressing, connect, release, flow control, multiplexing, error control, crash recovery

**TCP**: connection-oriented, reliable, ordered, flow + congestion control, 20–60 B header, unicast. HTTP, FTP, SMTP, SSH.
**UDP**: connectionless, best effort, 8 B header (src port, dst port, length, checksum), fast, broadcast/multicast. DNS, DHCP, VoIP, TFTP.
**Reliability**: seq numbers, ACK = next byte expected, retransmit on timeout / 3 dup ACKs, checksum, sliding window.
**Handshake**: SYN ($x$) $\rightarrow$ SYN+ACK ($y$, $x$+1) $\rightarrow$ ACK ($y$+1). **Close**: FIN, ACK, FIN, ACK; TIME-WAIT 2 MSL.
**DNS** UDP 53: root $\rightarrow$ TLD $\rightarrow$ authoritative. Client$\rightarrow$resolver **recursive**; resolver$\rightarrow$servers **iterative** (referrals). Cache by TTL. A, AAAA, CNAME, MX, NS, PTR.
**TCP header**: src port, dst port, seq, ack, HLEN, flags (URG ACK PSH RST SYN FIN), window, checksum, urgent pointer, options.
**UDP header**: src port, dst port, length, checksum (8 B).
**Session layer**: dialog control, checkpoints (synchronisation), RPC.
**Presentation**: translation, encryption, compression.
**Say it**: TCP = phone call, UDP = posted letter.

**Ports**: FTP 20/21, SSH 22, Telnet 23, SMTP 25, DNS 53, DHCP 67/68, TFTP 69, HTTP 80, POP3 110, IMAP 143, SNMP 161, HTTPS 443.
**E-mail**: UA $\rightarrow$ SMTP $\rightarrow$ MTA $\rightarrow$ SMTP $\rightarrow$ mailbox $\rightarrow$ POP3 (download, delete) / IMAP (stay, sync). MIME for attachments.
**HTTP** 80 / HTTPS 443: request–response, GET/POST, stateless, 200 OK / 404.
**FTP**: control 21 (whole session), data 20 (per file); active vs passive.
**Telnet** 23 plain text, **SSH** 22 encrypted; NVT.
**RPC**: call $\rightarrow$ client stub marshals $\rightarrow$ network $\rightarrow$ server stub unmarshals $\rightarrow$ procedure; result back.
**Compression**: lossless RLE (4A3B), Huffman (join 2 smallest, short codes for frequent), LZW (dictionary); lossy JPEG, MPEG, MP3.
**Troubleshoot**: ask $\rightarrow$ cable $\rightarrow$ `ipconfig /all` (169.254 = DHCP fail) $\rightarrow$ ping 127.0.0.1 $\rightarrow$ ping gateway $\rightarrow$ ping 8.8.8.8 / tracert $\rightarrow$ nslookup $\rightarrow$ browser.
**Commands**: ipconfig /all, hostname, ping, tracert (TTL), netstat -an, nslookup, arp -a.
**Cables**: unlike = straight (B–B); like = crossover (B–A, 1$\leftrightarrow$3, 2$\leftrightarrow$6). T568B: W-Or, Or, W-Gr, Bl, W-Bl, Gr, W-Br, Br.